

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

תאריך: 23.1.2005
משך הבחינה: שלוש שעות

מועד א' - סמסטר א' - תשס"ה – 2004-2005
המורה: פרופ' ג'ף רוננשיין

מילוי פרטים אישיים:

מספר מחברת: _____

מספר ת"ז (9 ספרות): _____

הנחיות:

1. בתום הבחינה יש להגיש שאלון זה יחד עם מחברת הבחינה.
2. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
3. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
4. יש להתחיל את התשובה לכל שאלה בעמוד חדש, ולציין בראש העמוד בצורה ברורה את מספר השאלה.
5. יש למחוק טיוטות באופן ברור.
6. יש להשאיר שוליים.
7. אסור להשתמש בעט אדום.
8. הקפידו על סגנון תכנותי נאות: כתבו באופן מודולרי, אל תכפילו קוד וכו'.
9. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
10. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה, גם אם היא נכונה!
11. אין להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בקלט (למשל מותר להקצות מערך בן תא אחד פעם אחת).

בחירת השאלות לבדיקה:

יש לסמן v ליד השאלות שברצונך שייבדקו: לכל היותר 3 סימני v; לא תיבדקנה שאלות שלא מסומן לידן v.

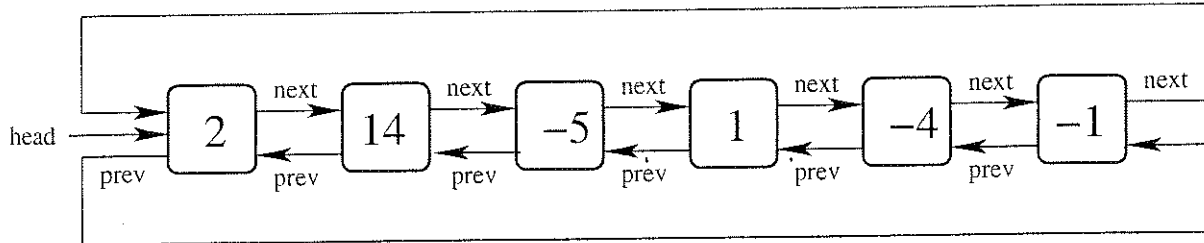
מספר שאלה	סמן v ליד שאלה שברצונך שתיבדק	לשימוש הבודק
1		
2		
3		
4		

שאלה 1

בתכנית מוגדרת רשימה משורשרת דו כיוונית מעגלית, באמצעות המבנה:

```
public class Node {
    public int data;
    public Node next;
    public Node prev;
}
```

להלן דוגמא לרשימה כנ"ל:

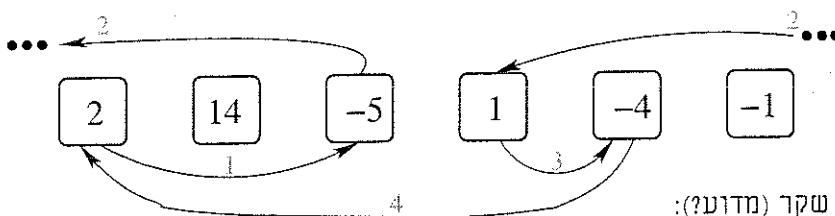


נגדיר דילוג ברשימה מעגלית באופן הבא: יהי p מצביע ל-Node

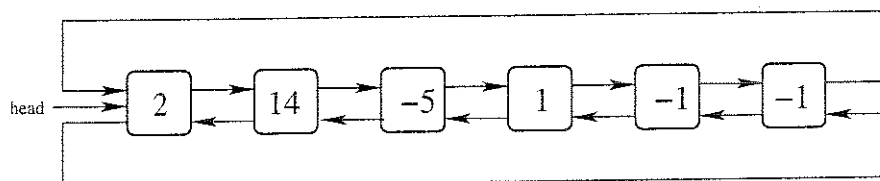
- אם $p.data$ חיובי אזי יש לבצע $p.data$ תנועות באמצעות המצביע $next$. לדוגמא, ברשימה הנ"ל, דילוג מהתא השמאלי (שבו $p.data == 2$) יעביר אותנו שני תאים ימינה לתא שתוכנו -5.
- אם $p.data$ שלילי אזי יש לבצע $p.data$ תנועות באמצעות המצביע $prev$. נשים לב שכיוון שהרשימה מעגלית, דילוג מהתא המכיל -5 יוביל אותנו לתא שמכיל 1.
- אם $p.data$ אפס לא נווע כלל (תוצאת הדילוג איננה משנה את p).

נגדיר מסלול ברשימה מעגלית כסידורה של דילוגים עוקבים.

א. כתוב פונקציה סטטית בשם `hasPath` (השייכת למחלקה ללא `data-members`) אשר מקבלת מצביע (מטיפוס `Node`) לראשה של רשימה מעגלית לא ריקה כנ"ל, ומחזירה אמת אם קיים מסלול שמתחיל בראש הרשימה וחוזר לראש הרשימה (23 נק'). לדוגמא, בעבור הרשימה הנ"ל, הפונקציה תחזיר אמת (`true`) שכן קיים מסלול כנדרש (המספרים ליד החצים מראים את סדר הדילוגים):



להלן דוגמא בעבורה הפונקציה תחזיר שקר (מדוע?):



הנחיות:

- בסעיף זה אסור להקצות זיכרון (כלומר אסור להשתמש ב-`new`).
- מותר להוסיף שדות ופונקציות ל-`Node` (כל עוד כתבתם בצורה ברורה מהי התוספת).

ב. מהו סדר גודל זמן הריצה של הפונקציה שכתבתם? (המקרה הגרוע). הסבר. (5 נק')

ג. תארו במילים כיצד הייתם פותרים את סעיף א' אילו מותר היה להקצות זיכרון, אך אסור היה לשנות את `Node`. שימו לב: בסעיף זה אינכם נדרשים לכתוב קוד, עליכם לתאר את הפתרון המוצע באופן מילולי (ובשום אופן לא יותר מחצי עמוד). (5 נק')

שאלה 2

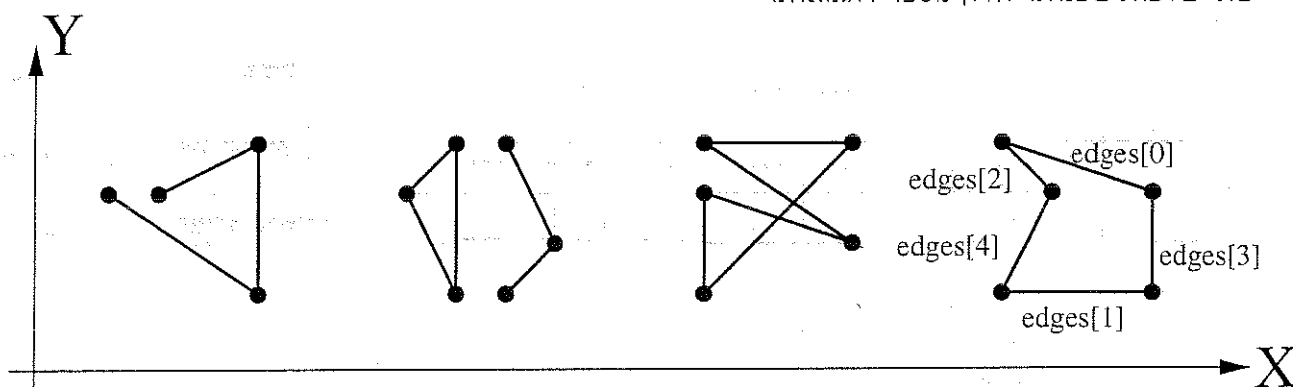
נתונות שתי מחלקות המייצגות נקודה וצלע בסריג דו מימדי:

```
public class Point {
    public int x;
    public int y;
    public boolean equals(Point p) { return x == p.x && y == p.y; }
}

public class Edge {
    public Point p1;
    public Point p2;
    public static boolean intersect(Edge e1, Edge e2) {
        // assume this method is implemented.
    }
}
```

כאשר Edge מייצגת את הצלע שעוברת בין הנקודות p_1 ו- p_2 , והפונקציה `intersect` מחזירה אמת אם שתי הצלעות הנתונות e_1 ו- e_2 נחתכות (צלעות נחתכות אם יש להן נקודה משותפת כלשהי, גם אם נקודה זאת היא p_1 או p_2).

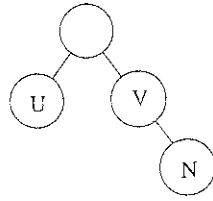
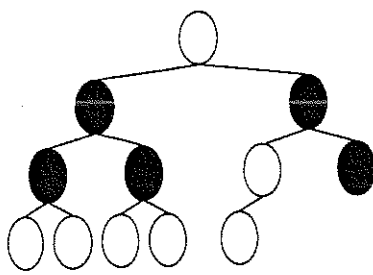
א. ממשו פונקציה סטטית בשם `isPolygon` המקבלת מערך בשם `edges` של צלעות, ומחזירה אמת אם שתי הצלעות הנתונות מהוות פוליגון יחיד. פוליגון מוגדר להיות: רצף של צלעות שבו לכל צלע e קיימות שתי צלעות שכנות (שונות מ- e וזו מזו) המתחברות ל- e בנקודות הקצה שלה ($e.p_1$ ו- $e.p_2$). רצף הצלעות הנוצר סוגר מעגל. מלבד נקודות ההתחברות, אסור ששתי צלעות תיגענה זו בזו. לכל צלע יש בדיוק שתי צלעות שכנות. להלן מספר דוגמאות:



מבין הנ"ל, רק אוסף הצלעות הימיני מהווה פוליגון. שימו לב שצלעות שכנות במעגל אינן בהכרח שכנות בפוליגון (כלומר לא ניתן להניח דבר לגבי סדר הצלעות ב-`edges` ולכן יתכן שאוסף הצלעות ב-`edges` מהווה פוליגון גם אם הצלעות $edges[i]$ ו- $edges[i+1]$ אינן שכנות). שאלה זו מותר (אם כי לא הכרחי) לפתור באמצעות הקצאת מבנה נתונים חדש, בנוסף למערך שקיבלתם. (28 נק')

ב. מהו סדר גודל זמן הריצה של הפונקציה שכתבת? (המקרה הגרוע). הסבר. (5 נק')

שאלה 3



- קודקוד U בעץ בינארי מהווה דוד אם יש לו אחיין, כלומר קיים קודקוד V ($U \neq V$) כך של- U, V יש אב משותף וכן ל- V יש לפחות ילד אחד.
- לדוגמא, העץ הימני הוא העץ הפשוט ביותר בו קיים קודקוד דוד (U הוא הדוד, N הוא האחיין).
- דוגמא נוספת: בעץ השמאלי יש בדיוק 5 קודקודי דוד (מסומנים בשחור).
- בתוכניתנו, עץ בינארי מוגדר באמצעות המבנה:

```
public class Node {
    public Object data;
    public Node left;
    public Node right;
}
```

- א. כתוב פונקציה סטטית בשם `uncle` (השייכת למחלקה ללא `data-members`) שמקבלת שורש של עץ בינארי (`Node`) כנ"ל, ומחזירה את מספר קודקודי הדוד בעץ. כריגיל, מותר להגדיר פונקציות עזר. (28 נק')
- ב. מהו סדר גודל זמן הריצה של הפונקציה שכתבת? (המקרה הגרוע). הסבר. (5 נק')

שאלה 4

כל סעיפי שאלה זו מתייחסים לשפת התכנות Java. ענה על 8 מתוך 10 הסעיפים הבאים. משקל כל הסעיפים זהה. בכל סעיף יש להקיף תשובה אחת בלבד (אם שיניתם תשובתכם יש למחוק את התשובה הקודמת באופן ברור שאינו משתמע לשני פנים).

מונחים בהם נעשה שימוש בשאלה זו:

מינוח:	מצביע	מחלקה	ממשק	חבילה	העמסה	החלפה	בנאי	המרה	פונקציה
פירוש:	reference	class	interface	package	overload	override	constructor	cast	method

- א. נניח שהמחלקה `D` יורשת את המחלקה `B`, וברצוננו לממש את `D.f` (`f` היא פונקציה של `D` שאינה סטטית) כ-`override` על `B.f` (`f` היא פונקציה של `B` שאינה סטטית). איזו מבין הטענות הבאות אינה נכונה:
- על טיפוס ההחזרה של `D.f` להיות זהה לטיפוס ההחזרה של `B.f`.
 - על הארגומנטים של `D.f` להיות זהים לארגומנטים של `B.f`.
 - על ה-`exceptions` של `D.f` להיות זהים ל-`exceptions` של `B.f`.
 - על הרשאת הגישה (`private`, `public` וכו') של `D.f` להיות זהה (או פחות מגבילה) מהרשאת הגישה של `B.f`.
 - בחר/י בסעיף זה אם כל הטענות הנ"ל נכונות.

- ב. ע"פ עקרון ההחלפה של ליסקוב (`Liskov substitution principle`) איזו מבין הטענות הבאות נכונה:

- עדיף לא לממש ירושה כגון:
`SquareMatrix extends Matrix`
`Square extends Rectangle`
או כגון:
- עדיף שמחלקה תממש ממשק מאשר תירש ממחלקה אחרת, שכן ב-Java יש `multiple-interface` (אפשר לממש יותר מממשק אחד) אבל אין `multiple-inheritance` (אפשר לרשת מחלקה אחת בלבד), ולכן מימוש ממשק מאפשר החלפת `base-class` ביתר קלות.
- עדיף להגדיר `data-members` של מחלקה `A` כ-`private` (כלומר להפריד ממשק ממימוש), שכן אז ניתן להחליף את המימוש של `A` מבלי להשפיע על מחלקות אחרות (שמשתמשות ב-`A` דרך הממשק בלבד).
- עדיף לתכנת אלגוריתם כך שישתמש במצביע ל-`base-class`, שכן אז האלגוריתם יעבוד גם בעבור כל מחלקה שיורשת את ה-`base-class` (`polymorphism`).
- אין תשובה נכונה.

- ג. תהי γ מחלקה שמממשת שני ממשקים שונים: $I1$ ו- $I2$. בזמן ריצה, איזו מבין הטענות הבאות נכונה?
- (i) לעיתים נוכל להמיר מצביע מטיפוס $I1$ ל- γ באמצעות ההמרה לא מפורשת (implicit).
 - (ii) תמיד נוכל להמיר מצביע מטיפוס $I1$ ל- $I2$ באמצעות ההמרה מפורשת (explicit).
 - (iii) לעיתים נוכל להמיר מצביע מטיפוס γ ל- $I1$ אבל רק אם ההמרה מפורשת.
 - (iv) לעיתים נוכל להמיר מצביע מטיפוס $I2$ ל- $I1$ אבל רק אם ההמרה מפורשת.
 - (v) אין תשובה נכונה.

- ד. תהי f פונקציה אבסטרקטית במחלקה A . איזו מהטענות הבאות אינה נכונה?
- (i) כל מחלקה שיורשת את A חייבת לממש את f .
 - (ii) f איננה פרטית.
 - (iii) f איננה final.
 - (iv) בחר/י בסעיף זה אם כל הטענות הנ"ל נכונות.

ה. מה ניתן לומר על קטע הקוד הבא?

```
int f() { return g(); }
```

- (i) g לעולם לא תזרוק exception.
- (ii) ייתכן ש- g תזרוק exception אך f לעולם לא תזרוק exception.
- (iii) ייתכן מצב שבו f תזרוק exception, אך אין קשר בין עובדה זו לבין האם g עלולה לזרוק exception או לא.
- (iv) f תזרוק exception רק אם ערך ההחזרה של g איננו מטיפוס int.
- (v) אין תשובה נכונה.

ו. נניח f מכילה (בין היתר) את קטע הקוד הבא:

```
try {g();} catch(E e) {throw e;}
```

איזו מבין הטענות הבאות נכונה:

- (i) f לא תתקמפל.
- (ii) f לעולם לא תזרוק exception.
- (iii) אפשר לממש את f בפחות פקודות.
- (iv) E יורש את RuntimeException (או את אחד מצאצאיו).
- (v) אין תשובה נכונה.

ז. בהנתן מחלקה בשם Node המממשת עץ חיפוש:

```
class Node { int data;
             Node left, right;
             Node(int d) {data=d;} }
```

מה ניתן לומר על קטע הקוד הבא:

```
void f(Node n, int d) {
    if( n==null ) return;
    Node p = (d <= n.data) ? n.left : n.right;
    p = new Node(d);
}
```

- (i) כאשר n אינו null אז: f מוסיפה לו בן שמאלי (אם d קטן או שווה לערך המוחזק ע"י n) או ימני (אם d גדול מהערך המוחזק ע"י n).
- (ii) f עלולה לגרום לאובדן אחד מתתי העץ של n .
- (iii) סיבוכיות זמן הריצה של f היא $O(1)$.
- (iv) כל התשובות הנ"ל נכונות.
- (v) אין תשובה נכונה.

ח. בהינתן קטע הקוד הבא:

```
class A { A() { System.out.println(1); } }
class B extends A { B() { A a=new A(); System.out.println(2); } }
```

מה תדפיס הפקודה (הפלט נקרא משמאל לימין):

- 1 2 (i)
- 2 1 (ii)
- 1 (iii)
- 2 (iv)
- (v) אין תשובה נכונה.

ט. בהינתן קטע הקוד הבא:

```
class Shape { int f(Shape x) {return 0; } }
class Circle extends Shape { int f(Circle x) {return f(x)+1; } }
```

איזו מבין הטענות הבאות אינה נכונה:

- (i) f המוגדרת ב-Circle מהווה העמסה (overload) של f המוגדרת ב-Shape.
- (ii) f המוגדרת ב-Circle מחזירה 1.
- (iii) בעבור המצביע shp: כאשר מריצים את הפקודה:


```
Shape shp = new Circle();
shp.f(shp);
```

 אזי הפונקציה שנקראת היא f המוגדרת ב-Shape.
- (iv) לכל אובייקט מסוג Circle יש שתי פונקציות ובנאי.
- (v) בחר/י בסעיף זה אם כל הטענות הנ"ל נכונות.

י. בהינתן הקוד הבא:

```
public class C { int x; protected int y; }
```

נניח ש-C הוגדרה בתוך החבילה P. באילו מהאובייקטים הבאים הרשאת הגישה ל-x ו-y שונה בוודאות?

- אובייקטים שנצרו במחלקות השייכות ל-P.
 - אובייקטים שנצרו במחלקות השייכות לחבילות השונות מ-P.
 - אובייקטים מטיפוסים שירשו את C השייכים ל-P.
 - אובייקטים מטיפוסים שירשו את C השייכים לחבילות השונות מ-P.
- (i) הסעיף הראשון.
 - (ii) הסעיף הראשון והשני.
 - (iii) הסעיף השני.
 - (iv) הסעיף השלישי.
 - (v) הסעיף הרביעי.
 - (vi) הסעיף השני, השלישי והרביעי.

בהצלחה!